



Royco / PendleRoycoTrancheSY Audit Report

May 1, 2026





Table of Contents

Summary	2
Overview	3
Issues	4
[WP-I1] Risk advisory — <code>exchangeRate()</code> is a live pass-through to Royco's NAV computation; rate freshness depends on the deployed kernel	4
[WP-I2] Misleading comment on <code>exchangeRate()</code> — author's mental model of what <code>exchangeRate</code> represents is wrong	6
Appendix	8
Disclaimer	9



Summary

This report has been prepared for Royco / PendleRoycoTrancheSY smart contract, to discover issues and vulnerabilities in the source code of their Smart Contract as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.



Overview

Project Summary

Project Name	Royco / PendleRoycoTrancheSY
Codebase	https://github.com/roycoprotocol/royco-dawn
Commit	9750a33f5e3e8329b4f1ec449fd28e8e82b44c40
Language	Solidity

Audit Summary

Delivery Date	May 1, 2026
Audit Methodology	Static Analysis, Manual Review
Total Issues	2



[WP-I1] Risk advisory — `exchangeRate()` is a live pass-through to Royco's NAV computation; rate freshness depends on the deployed kernel

Informational

Issue Description

This is an informational risk advisory rather than a finding against the deployed code. The behaviour is intentional, but the surface it inherits from Royco varies by deployment.

`PendleRoycoTrancheSY.exchangeRate()` returns

`IRoycoVaultTranche(yieldToken).convertToAssets(PMath.ONE).nav` directly. The NAV value is computed live by the Royco kernel — it is not cached or smoothed at the SY layer. The freshness and manipulation properties of the rate are therefore inherited from whichever Royco kernel is deployed under the tranche.

Royco's kernel directory contains several kernel families with different rate sources:

- Admin-set / Chainlink-based kernels (e.g. `*ChainlinkToAdminOracle*`) read bounded oracle values; manipulation surface is whatever Chainlink + the admin-bound provides.
- Live share-price kernels (e.g. `Identical_ERC4626_ST_JT_SharePriceTo*_Kernel`, `IdenticalERC4626SharesOracleQuoter`) read ERC4626 `convertToAssets(1e18)` of an underlying vault.
- Live virtual-price kernels (e.g. `Identical_AA_IdleCDO_ST_JT_VirtualPriceOracle_Kernel`) read IdleCDO `virtualPrice()`.
- Pool exit-price kernels (e.g. `MaplePoolV2_ST_JT_ExitSharePriceToChainlinkOracle_Kernel`) read live pool state.

For deployments backed by a live share-price or virtual-price kernel, a downstream consumer that reads `SY.exchangeRate()` synchronously inside a transaction in which the underlying source is mid-callback or transiently distorted may receive a value that does not reflect the post-state.

This is structurally similar to the read-only-reentrancy class affecting Curve LP wrappers that read `get_virtual_price()` directly, but here it is one indirection layer deeper (consumer → SY → kernel → underlying source) and depends on the specific kernel choice.

```

28     function exchangeRate() public view override(PendleERC20SYUpV2) returns
    (uint256) {
29         // Royco tranche shares always have 18 decimals of precision (PMath.ONE ==
        1 whole tranche share)
30         // Return the exchange rate in NAV units (always has 18 decimals of
        precision)
31         return IRoycoVaultTranche(yieldToken).convertToAssets(PMath.ONE).nav;
32     }

```

Considerations for the deployment policy

- For each tranche the SY is deployed against, document which Royco kernel backs the NAV path.
- If a live-share-price or live-virtual-price kernel is in scope, the integrating Pendle market and any downstream NAV consumers should be aware that `exchangeRate()` reflects whatever the underlying source returns at that block — including transiently distorted values during callbacks into the underlying. (Potential read-only reentrancy.)
- Junior tranche wipeout is also worth flagging to consumers: `convertToAssets(1e18).nav` can decrease, and in extreme cases approach zero, when the kernel's accounting registers impermanent loss. Consumers that assume monotonically non-decreasing `exchangeRate` semantics should be reviewed against this.

No on-contract change is required by this advisory; the value of flagging it is to make the deployment-policy choice (which kernel allowlist?) and the downstream-integration assumption (live pass-through, not cached) explicit.

Status

✓ Fixed

[WP-I2] Misleading comment on `exchangeRate()` — author’s mental model of what `exchangeRate` represents is wrong

Informational

Issue Description

The `exchangeRate()` contains two comment lines that misrepresent how Pendle’s SY exchange-rate semantics work:

```
// Royco tranche shares always have 18 decimals of precision (PMath.ONE == 1 whole
tranche share)
// Return the exchange rate in NAV units (always has 18 decimals of precision)
```

The first line implies the implementation depends on tranche shares being 18-decimal — that `PMath.ONE` (`1e18`) corresponds to “1 whole tranche share” and would mis-scale otherwise. **That mental model is wrong.** Pendle’s SY interface defines `exchangeRate` as a *per-wei rate in 18-decimal fixed-point*: callers compute `assetValue = exchangeRate * balance / 1e18`, where `balance` is in SY-wei. The share-decimal terms cancel inside that formula — the implementation is correct for any tranche decimals, not just 18.

So the actual semantic of the existing code:

- `convertToAssets(1e18).nav` returns the NAV value of `1e18` SY-wei, in 18-decimal NAV.
- Pendle consumers compute `value = (NAV of 1e18 wei) * balance_wei / 1e18 = NAV of balance_wei` . ✓

The “always 18 decimals” framing in the comment is at best stylistic shorthand, at worst a future-bug seed: it suggests to a future maintainer that they need to special-case non-18-decimal tranches (they don’t), or alternatively that they could rely on `1e18 == 1 whole share` as an invariant (which only holds for 18-decimal yieldTokens). Either misunderstanding produces incorrect future code.

The risk is not on the deployed contract — the math is right. The risk is on **future maintenance**: anyone reading this comment to derive the contract’s correctness model will end up with the wrong model.

```
28     function exchangeRate() public view override(PendleERC20SYUpV2) returns
    (uint256) {
29         // Royco tranche shares always have 18 decimals of precision (PMath.ONE ==
        1 whole tranche share)
30         // Return the exchange rate in NAV units (always has 18 decimals of
        precision)
31         return IRoycoVaultTranche(yieldToken).convertToAssets(PMath.ONE).nav;
32     }
```

Recommendation

Remove the misleading comments at L29-30.

This is informational only — the current code is correct. The fix is to make the comment match the actual semantics so future readers do not derive the wrong invariants.

Status

 Acknowledged

Appendix

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by WatchPug; however, WatchPug does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication.



Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Smart Contract technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.